Lab. No. 05: seguridad en el ciclo de desarrollo

Propósito

Cada equipo realizará una evaluación de seguridad de su propio proyecto de software, incorporando controles automatizados y validando, en un entorno expresamente autorizado, al menos un riesgo técnico. El resultado esperado es un Pull Request de remediación y evidencia verificable del antes y después.

Resultados de aprendizaje

Al completar el laboratorio, el estudiante distingue SAST, DAST, SCA y detección de secretos; configura controles de seguridad en GitHub; prioriza riesgos en términos de confidencialidad, integridad y disponibilidad; valida un hallazgo sin realizar acciones destructivas; y demuestra una remediación con pruebas de regresión.

Condiciones del ejercicio

Trabajar exclusivamente sobre proyectos propios, repositorios autorizados y entornos de staging bajo control del equipo. No se permiten ataques a sistemas de terceros, pruebas activas no autorizadas, exfiltración, fuerza bruta, denegación de servicio o uso de datos personales reales. Usar usuarios y registros ficticios. No publicar credenciales ni evidencias sensibles.

Secuencia de ejecución

1. Establecer alcance y línea base

Registre URL del repositorio, tecnología, rama, commit SHA, responsables, flujos de autenticación y APIs. Identifique activos y el riesgo principal. Cree la rama security/lab-fsi y complete docs/SECURITY.md.

2. Activar controles nativos de GitHub

En Settings → Security / Advanced Security, habilite Dependabot alerts y, cuando esté disponible, CodeQL Default setup y secret scanning. Si el repositorio es privado, verifique disponibilidad del plan. Evidencie pantallas o ejecuciones sin compartir información sensible.

3. Integrar SAST y detección de secretos

Copie .github/workflows/seguridad-estatica.yml. Haga push de la rama, abra un Pull Request, revise Actions y descargue resultado-semgrep. Explique dos resultados y distinga falsas alarmas. Gitleaks puede requerir licencia para repositorios de organizaciones.

4. Revisar componentes y dependencias (SCA)

Revise el archivo de dependencias y Dependabot alerts. Seleccione una dependencia desactualizada o vulnerable y valide si realmente aplica a su código; registre versión, alcance y medida correctiva. No actualice a ciegas bibliotecas incompatibles.

5. Ejecutar DAST pasivo en staging propio

Despliegue un staging HTTPS aislado con datos ficticios. Configure en GitHub la variable STAGING_URL con la URL de un sistema propio autorizado. Copie .github/workflows/dast-staging.yml y ejecute manualmente el workflow. Descargue el informe ZAP. Si no hay staging, ejecute baseline local sobre localhost con Docker y documente la alternativa.

6. Validar manualmente un hallazgo

Formule una hipótesis de seguridad (por ejemplo: falta de encabezado CSP, cookie sin HttpOnly, control de acceso insuficiente en una ruta de prueba). Valide con peticiones inocuas, cuentas de laboratorio y sin consultar registros de terceros. Si no es reproducible, justifique por qué.

7. Corregir y reanalizar

Implemente una solución pequeña y rastreable; registre commit y Pull Request. Añada una prueba automatizada o un procedimiento de regresión. Reejecute SAST/SCA/DAST que correspondan y compare los resultados.

8. Presentar los resultados

Complete docs/REPORTE_HALLAZGOS.md y docs/SECURITY.md, comparta el enlace al Pull Request y prepare una demostración de cinco minutos: problema, impacto, evidencia, corrección, nueva evidencia y limitaciones.

Entregables requeridos

- Repositorio y commit de la línea base, enlace a la rama del laboratorio.
- Pipeline SAST + secretos funcionando o limitación técnica justificada.
- Evidencia SCA y al menos un reporte de DAST pasivo, o alternativa local documentada.
- Informe de mínimo dos hallazgos analizados, al menos uno corregido, con evidencia antes/después.
- Pull Request, prueba de regresión y exposición breve.

Evaluación

Criterio	Peso
Alcance, autorización y gestión del riesgo	15 %
Integración SAST, secretos y SCA	25 %
DAST y validación responsable	20 %
Corrección y nueva ejecución de pruebas	25 %
Informe y defensa técnica	15 %

Material de integración entregado

Se incluyen los workflows de GitHub Actions, el registro de seguridad, un checklist de Pull Request, el formato de reporte y el tablero interactivo HTML. Los estudiantes pueden copiar las carpetas .github/ y docs/ directamente a sus repositorios.

Referencias técnicas

GitHub Docs — Code scanning Default setup: https://docs.github.com/en/code-security/how-tos/find-and-fix-code-vulnerabilities/configure-code-scanning/configure-code-scanning

OWASP ZAP Baseline: https://www.zaproxy.org/docs/docker/baseline-scan/

Gitleaks Action: https://github.com/marketplace/actions/gitleaks